

DSM Partitioning

A Multi-Stage Pipeline for Reordering Dependency Structure Matrices

Simon Kraft Joshua Payne El Sall

CPSC 482/682 — Data Structures II
University of Northern British Columbia

April 7, 2026

Outline

- 1 Introduction
- 2 Methodology
- 3 Results
- 4 Conclusions

The challenge:

- Modern software & engineering projects involve **hundreds of interdependent tasks**
- Require coordination of large, multidisciplinary teams
- Traditional tools (Gantt, CPM) model sequential & parallel flows
- **fail for feedback dependencies**, common in complex product development

Goal

Find a **task ordering** that minimises conflicting information flows to enable faster, more predictable project execution.

Key observation

When task j supplies input to task i but is scheduled *after* it, assumptions must be made. If wrong, task i must be **re-executed**.

Design Structure Matrix (DSM)

Definition: A binary matrix $A \in \{0, 1\}^{n \times n}$
where

$$a_{ij} = 1 \iff \text{task } i \text{ depends on task } j$$

- Rows and columns labeled by the same task list
- Diagonal entries carry no inter-task meaning
- Equivalent to directed graph
 $G(A) = (V, E)$

Three dependency types:

- **Feed-Forward** ($j < i$): below diagonal — easy
- **Feed-Back** ($j > i$): above diagonal — problematic
- **Coupled**: mutual dependency — hardest

The Partitioning Problem

Objective: Find $P \in \{0, 1\}^{n \times n}$ such that $A' = P^T A P$ minimises:

Objective 1 — Feed-Back Marks

$$|\text{FBM}| = |\{a'_{ij} \neq 0 \mid j > i\}|$$

Minimise the **count** of above-diagonal nonzeros.

Objective 2 — Feedback Distance

$$\text{TFBD} = \sum_{a'_{ij} \in \text{FBM}} (j - i)$$

Minimise the **distance** of feed-back marks from the diagonal.

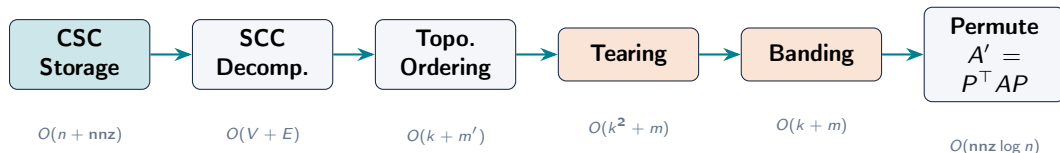
Best case

$G(A)$ has no directed cycles $\Rightarrow$ fully lower triangular,
 $|\text{FBM}| = 0$, $\text{TFBD} = 0$

General case

Achieve **Block Lower Triangular Form (BLTF)**: group cycle-participants into diagonal blocks and minimise feedback distance.

Our 6-Stage Pipeline



Data Structure: CSC

- Column pointer array `col_ptr[0...n]`
- Row index array `row_ind[0...nnz-1]`
- Storage: $O(n + \text{nnz})$
- No value array needed (binary DSM)

Why CSC?

Column j lists out-neighbours of vertex j , giving $O(\deg^+(j))$ access — exactly the pattern required for DFS-based SCC algorithms.

SCC Decomposition

Both algorithms implemented as **iterative DFS** over CSC, using an explicit stack (no recursion).

Tarjan's Algorithm

- **Single DFS pass**
- Each vertex v maintains $low(v)$: smallest reachable pre-order time
- $low(v) = v.pre$ at finish $\Rightarrow v$ is SCC root; pop stack
- $O(V + E)$ time, $O(V)$ space

Kosaraju-Sharir Algorithm

- **Two DFS passes**
- Pass 1: DFS on G , record finish order
- Pass 2: DFS on $rev(G)$ in reverse finish order; each DFS tree = one SCC
- $O(V + E)$ time, $O(V + E)$ space

After SCC decomposition, the **condensation DAG** is topologically sorted with **Kahn's algorithm** to determine the block ordering.

Tearing & Banding

Tearing (intra-block FBM reduction)

- Approximates the **minimum feedback arc set** within each non-trivial SCC
- Uses **Algorithm GR** (Eades et al., 1993): greedily removes sinks, then sources, then the vertex with maximum $\delta = d^+ - d^-$
- Backward edges in the returned ordering are torn
- $O(k^2 + m)$ per SCC (linear scan variant)

Banding (TFBD reduction)

- **Reverse Cuthill-McKee (RCM)**: concentrates nonzeros near diagonal
- Pseudoperipheral start via George-Liu BFS
- **Adjacent-swap refinement**: accepts swaps that decrease TFBD without increasing FBM; up to 20 passes
- Best of torn + RCM orderings is kept

Benchmark data

10 sparse matrices from the project repository, $n = 11$ to 27,770, $\text{nnz} = 4$ to 352,807. Compiled

Stage-wise Quality Improvement

Average reductions across 10 matrices:

Stage	FBM ↓	TFBD ↓
Baseline	—	—
TopoOnly	60.16%	86.56%
+ Tearing	76.47%	<i>slight</i> ↑
+ Banding	76.50%	85.95%

- Topo ordering provides the **largest single gain**
- Tearing drives further FBM reduction within SCCs
- Banding polishes TFBD without hurting FBM

Selected matrix results

Matrix	FBM ₀ → FBM _f	TFBD _f
Tina_AskCal	12 → 4	7
California	6063 → 192	669
wb-cs-stanford	20131 → 8537	1.78M
HEP-th-new	104435 → 8281	10.1M

Runtime Comparison: Tarjan vs. Kosaraju-Sharir

Key findings

- Both algorithms produce **identical SCC decompositions**
- Tarjan: mean $\approx 560 \mu\text{s}$
- Kosaraju-Sharir: mean $\approx 992 \mu\text{s}$
- Tarjan is $\approx 1.77\times$ faster on average
- Scaling compatible with expected $O(V + E)$ near-linear behaviour

Pipeline stage runtimes

Stage	Relative cost
Tarjan SCC	low
Kosaraju SCC	$1.77\times$ Tarjan
Condensation	low
Topo sort	low
Tearing	moderate
Banding	highest
Permutation	low

Why is Tarjan faster?

Single DFS pass vs. two passes + transpose graph construction in Kosaraju-Sharir.

Banding is the heaviest stage but delivers the final quality gains.

Qualitative Results — Matrix Spy Plots

California ($n = 9,664$)

Near-perfect partitioning: almost no residual FBM, no large SCC blocks visible.

wb-cs-stanford ($n = 9,914$)

One large persistent SCC block retains significant FBM far from diagonal — explains high residual TFBD.

Tina_AskCal ($n = 1,023$)

Clean block structure with well-separated components after reordering.

GD99_c ($n = 62$)

Illustrates the SCC structure clearly: many trivial singleton and two-node SCCs, plus two larger coupled blocks.

(See appendix figures for full spy plot comparisons before and after partitioning.)

Conclusions & Future Work

What we achieved

- Implemented full 6-stage DSM reordering pipeline in C++ with CSC
- Both Tarjan and Kosaraju-Sharir SCC algorithms — identical decompositions
- **76.50% average FBM reduction, 85.95% average TFBD reduction** across 10 benchmark matrices
- Tarjan $1.77\times$ faster than Kosaraju-Sharir
- Runtime scales near-linearly with $V + E$

Future directions

- **Better tearing:** replace greedy Algorithm GR with an exact or improved heuristic for minimum feedback arc set
- **Parallelism:** per-SCC tearing & banding are independent — parallelise for large matrices
- **Weighted DSMs:** extend to numerical dependency strengths; optimise weighted feedback cost

Acknowledgements & Team Contributions

AI Disclosure

Claude (Anthropic) and ChatGPT (OpenAI) were used to aid understanding of algorithmic concepts and literature (SCC decomposition, tearing, banding). No AI tool was used to generate code or write report text.

Team Member Contributions

Member	Contribution
Simon Kraft	[e.g., implemented Tarjan's algorithm; wrote Sections 2.1–2.3]
Joshua Payne	[e.g., implemented Kosaraju-Sharir; wrote Sections 2.4–2.5]
El Sall	[e.g., implemented tearing & banding; wrote Sections 3 & 4]

Thank you — questions welcome!